2. Consider the following *SDT*. Draw the **Dependency Graph** and show the values of the attribute offset, type, and width next to the appropriate non-terminal nodes of the following input string **float [5][2] y; int x;**

$P \rightarrow \quad \{ \ offset = 0; \ \} \ D$

$D \rightarrow T \ \mathbf{id} \ ; \quad \{ \ top.put(\mathbf{id}.lexeme, \ T.type, \ offset); \ offset = offset + T.width; \ \} \ D_1$

$D \rightarrow \epsilon$

$T \rightarrow B \ \{ \ t = B.type; \ w = B.width; \ \} \ C$ {T.type = C.type; T.width = C.width}

$B \rightarrow \mathbf{int} \quad \{ \ B.type = integer; \ B.width = 4; \ \}$

$B \rightarrow \mathbf{float} \quad \{ \ B.type = float; \ B.width = 8; \ \}$

$C \rightarrow \epsilon \quad \{ \ C.type = t; \ C.width = w; \ \}$

$C \rightarrow [\ \mathbf{num} \ ] \ C_1$ { C.type = $array(\mathbf{num}.value, \ C_1.type); \ C.width = \mathbf{num}.value \times C_1.width; \ \}$

2. Consider the following *SDT*. Draw the **Dependency Graph** and show the values of the attribute offset, type, and width next to the appropriate non-terminal nodes of the following input string **float [5][2] y; int x;**

$P \rightarrow \quad \{ \; offset = 0; \; \} \; D$

$D \rightarrow T \; \mathbf{id} \; ; \quad \{ \; top.put(\mathbf{id}.lexeme, \; T.type, \; offset); \; offset = offset + T.width; \; \} \; D_1$

$D \rightarrow \epsilon$

$T \rightarrow B \, \{ \; t = B.type; \; w = B.width; \; \} \; C$ {T.type = C.type; T.width = C.width}

$B \rightarrow$ **int** $\quad \{ \; B.type = integer; \; B.width = 4; \; \}$

$B \rightarrow$ **float** $\quad \{ \; B.type = float; \; B.width = 8; \; \}$

$C \rightarrow \epsilon \quad \{ \; C.type = t; \; C.width = w; \; \}$

$C \rightarrow$ [**num**] $C_1 \quad$ { C.type = $array(\mathbf{num}.value, \; C_1.type); \; C.width = \mathbf{num}.value \times C_1.width;$ }

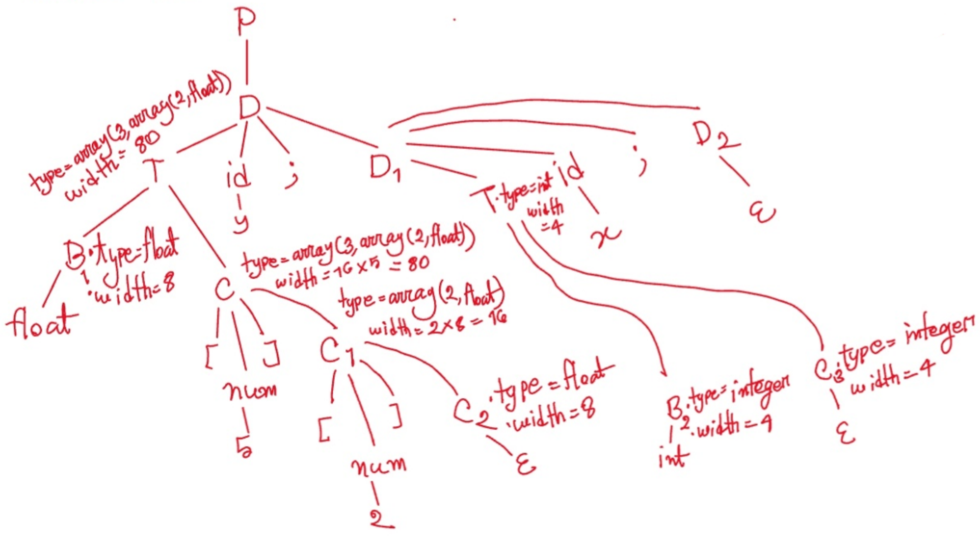
float [5] [2] y ; int x ;

offset	t	w
~~0~~	~~float~~	8
~~80~~	integer	4
84		

top

y	array(3, array(2, float))	0
x	integer	80

Parse tree



3. Construct the **Three Address Code** from the following code:

x = ((p-q)*(-r)+(p+r))-(q*p)+r;
generateRandom(seed, seedMod2, seedMod5);

t_1 = minus q

$t_2 = p + t_1$

t_3 = minus r

$t_4 = t_2 * t_3$

$t_5 = p + r$

$t_6 = t_4 + t_5$

$t_7 = q * p$

t_8 = minus t_7

$t_9 = t_6 + t_8$

$t_{10} = t_9 + r$

$x = t_{10}$

param seed
param seedMod2
param seedMod5

call generateRandom, 3